

### Exercici 3.1: Tradueix el codi anterior a SISA-F:

#### CODI EN ALT NIVELL:

```
main(){
    int i;
    int X[]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int Y[]={-2, 0, 3, 6, -2, 12, -3, 14, 5, 6};
    int Z[10];
    for(i=0; i<10;i++)
    {
        Z[i]=X[i]+Y[i];
    }
}

.include "macros.s"
.include "crt0.s"
.data
    X: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
    Y: .word -2, 0, 3, 6, -2, 12, -3, 14, 5, 6
    Z: .fill 10,2,0
.text
main:
    MOVI R0, 0 ;i=0
while:
    MOVI R4, 20
    $CMPLT R4,R0,R4 ;i < 20? SI? fem bucle, en cas contrari halt
    BZ R4, fi

    $MOVEI R1, X ;r1=@X[0]
    ADD R1,R1,R0 ;r1=@X[i]
    LD R1, 0(R1) ;r1=X[i]

    $MOVEI R2, Y ;r2=@Y[0]
    ADD R2,R2,R0 ;r2=@Z[i]
    LD R2, 0(R2) ;r2=Y[i]

    $MOVEI R3, Z ;r3=@Z[0]
    ADD R3,R3,R0 ;r3=@Z[i]

    ADD R5,R1,R2 ;r5=X[i]+Y[i]
    ST 0(R3), R5 ;Z[i]=X[i]+Y[i]

    ADDI R0,R0,2 ;i++
    BNZ R4, while

fi:
    halt
```

| [2][ | Data: hexadecimal byte ]                        |                  |
|------|-------------------------------------------------|------------------|
| 0050 | c5 c5 c4 c7 c3 c9 c2 cb c1 cd c0 cf 2c f2 6c f4 | , l              |
| 0060 | ec f6 fa 2f c2 43 c1 45 c0 47 ac f2 0f 54 42 16 | / C E G TB       |
| 0070 | 07 66 14 54 00 55 61 02 a1 04 80 34 84 ac 07 6d | f T Ua 4 m       |
| 0080 | 24 54 00 55 28 f2 61 02 a1 04 80 34 84 ac c0 37 | \$T U( a 4 7     |
| 0090 | c1 35 c2 33 c6 2f f0 f6 b0 f2 70 f0 c0 bf c1 bd | 5 3 / p          |
| 00a0 | c2 bb c3 b9 c4 b7 c5 b5 c6 b3 c7 b1 d0 2f c0 3d | / =              |
| 00b0 | c1 3b c2 39 c3 37 c4 35 c5 33 c6 31 ce 2f 24 f0 | ; 9 7 5 3 1 /\$  |
| 00c0 | 31 32 33 34 35 36 37 38 39 30 08 51 57 45 52 54 | 1234567890 QWERT |
| 00d0 | 59 55 49 4f 50 2b 41 53 44 46 47 48 4a 4b 4c 0a | YUIOP+ASDFGHJKL  |
| 00e0 | 5a 58 43 56 42 4e 4d 2c 2e 2d 20 00 00 50 14 58 | ZXCVBNM,.- P X   |
| 00f0 | 04 18 0f 68 14 52 01 53 60 02 40 32 28 54 01 55 | h R S` @2(T U    |
| 0100 | a0 04 80 34 3c 56 01 57 e0 06 62 0a c0 4a 02 20 | 4<V W b J        |
| 0110 | ee 69 ff ff 01 00 02 00 03 00 04 00 05 00 06 00 | i                |
| 0120 | 07 00 08 00 09 00 0a 00 fe ff 00 00 03 00 06 00 |                  |
| 0130 | fe ff 0c 00 fd ff 0e 00 05 00 06 00 ff ff 02 00 |                  |
| 0140 | 06 00 0a 00 03 00 12 00 04 00 16 00 0e 00 10 00 |                  |

### Exercici 3.2:

Quant els tinguis tradit comprova en el simulador les adreces assignades als vectors X, Y i Z. Amb l'adreça inicial de X, Y, Z dedueix les adreces de:

@X[0]=0x0114 @Y[0]=0x0128 @Z[0]=0x013C  
 @X[1]=0x0116 @Y[1]=0x012A @Z[1]=0x013E  
 @X[2]=0x0118 @Y[2]=0x012C @Z[2]=0x0140  
 @X[3]=0x011A @Y[3]=0x012E @Z[3]=0x0142  
 @X[4]=0x011C @Y[4]=0x0130 @Z[4]=0x0144  
 @X[5]=0x011E @Y[5]=0x0132 @Z[5]=0x0146  
 @X[6]=0x0120 @Y[6]=0x0134 @Z[6]=0x0148  
 @X[7]=0x0122 @Y[7]=0x0136 @Z[7]=0x014A  
 @X[8]=0x0124 @Y[8]=0x0138 @Z[8]=0x014C  
 @X[9]=0x0126 @Y[9]=0x013A @Z[9]=0x014E

**Repàs: Quants bits tenen les adreces en SISA-F?**

Cada adreça en SISA-F té 8 bits.

**Activitat 3.B:** Volem que el modificar el codi anterior per tal de que ens mostri per pantalla la direcció de línia de cache en cadascún dels accesos a cache, tant de X, Y i Z. Obtindrem també el tag (etiqueta) i l'observarem al simulador encara que no el mostrem per pantalla. Suposarem que la cache que tindria el processador és una cache directa de 512 línies, i on cada línia són 4 paraules (Recorda que a més a més 1 paraula son 2 bytes en SISA-F). Tenint en compte aquestes dades, completa el següent pseudo-codi en alt nivell:

```
main(){
    int i, line, line_cache, tag, address, j;
    char character;
    int fil=4, col;
    int X[]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int Y[]={-2, 0, 3, 6, -2, 12, -3, 14, 5, 6};
    int Z[10];
    for(i=0; i<10;i++) {
        Z[i]=X[i]+Y[i];
        col=15;
        /*afegeix una o més instruccions que obtinguin l'adreça de X[i] i la posi a la variable address */
        _address = (int)&X[i];_____ ;
        line= _address>>3; /*obtenim la línia o bloc de memoria principal */
        line_cache= line &__0x1FF_____; /*completa posant la màscara adequada per obtenir */
        /* el número de línia a memoria cache */
        tag= _address>>____12____; /*completa per obtenir el tag */
        /*Ara treurem el valor de line_cache en binari per pantalla, per això ho haurem de convertir a ASCII, bit a bit */
        out(Rfil_pant, fil);
        while(line_cache!=0){
            character= line_cache&1; /*obtenim el bit de menys pes */
            character=character +__'0'__; /*completa que hem de sumar per transformar-lo en ASCII*/
            5
            out(Rcol_pantalla, col);
            out(Rdat_pant, character);
            out(Rcont_pantalla, 0x8000);
            line_cache=line_cache>>__1__; /*completa per desplaçar un bit */
            col--;
        }
        col=30; /*posem al columna a 30 */
        /*repetim el mateix per line_cache de Y[i] */
        /*afegeix una o més instruccions que obtinguin l'adreça de Y[i] i la posi a la variable address */
    }
}
```

```

        ____address = (int)&Y[i];____;
        .
        .
        .
        col=45; /*posem la columna a 45 */
        /*repetim el mateix per line_cache de Z[i] */
        /*afegeix una o més instruccions que obtinguin l'adreça de Z[i] i la posi
        a la variable
        address */
        ____address = (int)&Z[i];____;
        .
        .
        .
        /* per la següent iteració del bucle en i incrementem la fila */
        fil++;
    }
}

```

### **CODI EN ALT NIVELL COMPLET:**

```

main() {
    int i, line, line_cache, tag, address, j;
    char character;
    int fil = 4, col;
    int X[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int Y[] = {-2, 0, 3, 6, -2, 12, -3, 14, 5, 6};
    int Z[10];

    for (i = 0; i < 10; i++) {
        Z[i] = X[i] + Y[i];
        col = 15;

        /* Adreça de X[i] */
        address = (int)&X[i];
        line = address >> 3; // Obtenim la línia o bloc de memòria principal
        line_cache = line & 0x1FF; // Màscara per obtenir la línia de memòria cau
        tag = address >> 12; // Obtenim el tag

        /* Mostrar line_cache en binari */
        out(Rfil_pant, fil);
        while (line_cache != 0) {
            character = line_cache & 1; // Obtenim el bit menys significatiu
            character = character + '0'; // Convertim a ASCII
            out(Rcol_pantalla, col);
            out(Rdat_pant, character);
            out(Rcont_pantalla, 0x8000);
            line_cache = line_cache >> 1; // Desplacem un bit
            col--;
        }
    }
}

```

```
}
```

```
col = 30; // Posem la columna a 30
```

```
/* Adreça de Y[i] */  
address = (int)&Y[i];  
line = address >> 3;  
line_cache = line & 0x1FF;  
tag = address >> 12;
```

```
// Repetim el procés per line_cache de Y[i]  
out(Rfil_pant, fil);  
while (line_cache != 0) {  
    character = line_cache & 1;  
    character = character + '0';  
    out(Rcol_pantalla, col);  
    out(Rdat_pant, character);  
    out(Rcont_pantalla, 0x8000);  
    line_cache = line_cache >> 1;  
    col--;  
}
```

```
col = 45; // Posem la columna a 45
```

```
/* Adreça de Z[i] */  
address = (int)&Z[i];  
line = address >> 3;  
line_cache = line & 0x1FF;  
tag = address >> 12;
```

```
// Repetim el procés per line_cache de Z[i]  
out(Rfil_pant, fil);  
while (line_cache != 0) {  
    character = line_cache & 1;  
    character = character + '0';  
    out(Rcol_pantalla, col);  
    out(Rdat_pant, character);  
    out(Rcont_pantalla, 0x8000);  
    line_cache = line_cache >> 1;  
    col--;  
}
```

```
/* Per a la següent iteració del bucle en i, incrementem la fila */  
fil++;
```

```
}
```

```
}
```

#### Exercici 4.1: Tradueix el codi anterior a SISA-F i simula'l pas a pas.

Codi en sisa-f:

```
.include "macros.s"
.include "crt0.s"

.data
    X: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
    Y: .word -2, 0, 3, 6, -2, 12, -3, 14, 5, 6
    Z: .fill 10,2,0

.text
main:
    movi R0, 0          ; i = 0
    movi R6, 4          ; fil = 4

for:
    movi R4, 20         ; r4 = 20
    $CMPLT R4, R0, R4    ; Comparamos si i < 20
    BZ R4, fi           ; Si no, salimos del bucle

    ; Procesamos X[i]
    $movei R1, X         ; R1 = @X[0]
    ADD R1, R1, R0       ; R1 = @X[i]

    ; Calculamos line_cache y tag de X[i]
    $MOVEI R7, -3       ; r7 = 3
    SHL R2, R1, R7      ; line = R2 = @X[i] >> 3
    $MOVEI R7, 0x1FF    ; r7 = 0x1FF
    AND R3, R2, R7      ; line_cache = line & 0x1FF (mascara de 9 bits)
    $MOVEI R7, -12      ; r7 = 12
    SHL R2, R1, R7      ; tag = @X[i] >> 12

    ; Mostrar line_cache en binario para X[i]
    $MOVEI R7, 15       ; col = 15
    out Rfil_pant, R6    ; fila = fil

mostrar_X:
    $MOVEI R1, 0
    $CMPEQ R1, R3, R1    ; line_cache != 0? seguimos si es diferente
    BNZ R1, procesar_Y

    $MOVEI R2, 1
    AND R5, R3, R2      ; Obtenemos el bit de menor peso
    $MOVEI R2, '0'
    ADD R5, R5, R2      ; Convertimos el bit a caracter ASCII ('0' o '1')

    $movei R2, 0x8000 ;posada en marxa
```

```

out Rcol_pant, R7 ;col = R7
out Rdat_pant, R5
out Rcon_pant, R2

$movei R2, -1
SHL R3, R3, R2      ; Desplazamos a la derecha para obtener el siguiente bit

ADDI R7,R7,-1      ; col--
BZ R1, mostrar_X

; Procesamos Y[i]
procesar_Y:
$movei R1, Y        ; R1 = @Y[0]
ADD R1, R1, R0      ; R1 = @Y[i]

; Calculamos line_cache y tag de X[i]
$MOVEI R7, -3       ; r7 = 3
SHL R2, R1, R7      ; line = R2 = @Y[i] >> 3
$MOVEI R7, 0x1FF    ; r7 = 0x1FF
AND R3, R2, R7      ; line_cache = line & 0x1FF (mascara de 9 bits)
$MOVEI R7, -12      ; r7 = 12
SHL R2, R1, R7      ; tag = @Y[i] >> 12

; Mostrar line_cache en binario para Y[i]
$MOVEI R7, 30       ; col = 30
out Rfil_pant, R6    ; fila = fil
mostrar_Y:
$MOVEI R1, 0
$CMPEQ R1,R3,R1      ; line_cache != 0? seguimos si es diferente
BNZ R1, procesar_Z

$MOVEI R2, 1
AND R5, R3, R2      ; Obtenemos el bit de menor peso
$MOVEI R2, '0'
ADD R5,R5,R2        ; Convertimos el bit a caracter ASCII ('0' o '1')

$movei R2, 0x8000 ;posada en marcha
out Rcol_pant, R7 ;col = R7
out Rdat_pant, R5
out Rcon_pant, R2

$movei R2, -1
SHL R3, R3, R2      ; Desplazamos a la derecha para obtener el siguiente bit

ADDI R7,R7,-1      ; col--
BZ R1, mostrar_Y

; Preparamos Z[i]

```

procesar\_Z:

```
$movei R1, Z      ; R1 = @Z[0]
ADD R1, R1, R0    ; R1 = @Z[i]
```

```
$movei R2, X      ; R2 = @X[0]
ADD R2, R1, R0    ; R2 = @X[i]
LD R2, 0(R1)      ; R2 = X[i]
```

```
$movei R3, Y      ; R3 = @Y[0]
ADD R3, R3, R0    ; R3 = @Y[i]
LD R3, 0(R3)      ; R3 = Y[i]
```

```
; Operacion: Z[i] = X[i] + Y[i]
ADD R2, R2, R3    ; R2 = X[i] + Y[i]
ST 0(R1), R2      ; Almacenamos Z[i] = X[i] + Y[i]
```

```
; Mostrar line_cache y tag de Z[i]
$MOVEI R7, -3     ; r7 = 3
SHL R2, R1, R7    ; line = R2 = @Z[i] >> 3
$MOVEI R7, 0x1FF  ; r7 = 0x1FF
AND R3, R2, R7    ; line_cache = line & 0x1FF (mascara de 9 bits)
$MOVEI R7, -12    ; r7 = 12
SHL R2, R1, R7    ; tag = @Z[i] >> 12
```

```
; Mostrar line_cache en binario para Z[i]
```

```
$MOVEI R7, 45     ; col = 45
out Rfil_pant, R6 ; fila = fil
```

mostrar\_Z:

```
$MOVEI R1, 0
$CMPEQ R1,R3,R1    ; line_cache != 0? seguimos si es diferente
BNZ R1, seguir
```

```
$MOVEI R2, 1
AND R5, R3, R2    ; Obtenemos el bit de menor peso
$MOVEI R2, '0'
ADD R5,R5,R2      ; Convertimos el bit a caracter ASCII ('0' o '1')
```

```
$movei R2, 0x8000 ;posada en marcha
out Rcol_pant, R7 ;col = R7
out Rdat_pant, R5
out Rcon_pant, R2
```

```
$movei R2, -1
SHL R3, R3, R2    ; Desplazamos a la derecha para obtener el siguiente bit
```

```
ADDI R7,R7,-1    ; col--
BZ R1, mostrar_Z
```



```

; Iteracion siguiente
seguir:
    ADDI R0, R0, 2      ; i++
    ADDI R6, R6, 1      ; fil++
    BNZ R4, for         ; Repetir el bucle si i < 10

fi:
    halt                ; Fin del programa

```

### Simulació pas a pas:

```

x[0]=0x01E4 -> line=0x003C -> line_cache=0x003C -> tag=0 -> se imprime -> 111100
x[1]=0x01E6 -> line=0x003C -> line_cache=0x003C -> tag=0 -> se imprime -> 111100
x[2]=0x01E8 -> line=0x003D -> line_cache=0x003D -> tag=0 -> se imprime -> 111101
x[3]=0x01EA -> line=0x003D -> line_cache=0x003D -> tag=0 -> se imprime -> 111101
x[4]=0x01EC -> line=0x003D -> line_cache=0x003D -> tag=0 -> se imprime -> 111101
x[5]=0x01EE -> line=0x003D -> line_cache=0x003D -> tag=0 -> se imprime -> 111101
x[6]=0x01F0 -> line=0x003E -> line_cache=0x003E -> tag=0 -> se imprime -> 111110
x[7]=0x01F2 -> line=0x003E -> line_cache=0x003E -> tag=0 -> se imprime -> 111110
x[8]=0x01F4 -> line=0x003E -> line_cache=0x003E -> tag=0 -> se imprime -> 111110
x[9]=0x01F6 -> line=0x003E -> line_cache=0x003E -> tag=0 -> se imprime -> 111110

```

```

y[0]=0x01F8 -> line=0x003F -> line_cache=0x003F -> tag=0 -> se imprime -> 111111
y[1]=0x01FA -> line=0x003F -> line_cache=0x003F -> tag=0 -> se imprime -> 111111
y[2]=0x01FC -> line=0x003F -> line_cache=0x003F -> tag=0 -> se imprime -> 111111
y[3]=0x01FE -> line=0x003F -> line_cache=0x003F -> tag=0 -> se imprime -> 111111
y[4]=0x0200 -> line=0x0040 -> line_cache=0x0040 -> tag=0 -> se imprime -> 1000000
y[5]=0x0202 -> line=0x0040 -> line_cache=0x0040 -> tag=0 -> se imprime -> 1000000
y[6]=0x0204 -> line=0x0040 -> line_cache=0x0040 -> tag=0 -> se imprime -> 1000000
y[7]=0x0206 -> line=0x0040 -> line_cache=0x0040 -> tag=0 -> se imprime -> 1000000
y[8]=0x0208 -> line=0x003E -> line_cache=0x0042 -> tag=0 -> se imprime -> 1000001
y[9]=0x020A -> line=0x003E -> line_cache=0x0042 -> tag=0 -> se imprime -> 1000001

```

```

z[0]=0x020C -> line=0x003F -> line_cache=0x0042 -> tag=0 -> se imprime -> 1000001
z[1]=0x020E -> line=0x003F -> line_cache=0x0042 -> tag=0 -> se imprime -> 1000001
z[2]=0x0210 -> line=0x003F -> line_cache=0x0044 -> tag=0 -> se imprime -> 1000010
z[3]=0x0212 -> line=0x003F -> line_cache=0x0044 -> tag=0 -> se imprime -> 1000010
z[4]=0x0214 -> line=0x003F -> line_cache=0x0044 -> tag=0 -> se imprime -> 1000010
z[5]=0x0216 -> line=0x003F -> line_cache=0x0044 -> tag=0 -> se imprime -> 1000010
z[6]=0x0218 -> line=0x003F -> line_cache=0x0046 -> tag=0 -> se imprime -> 1000011
z[7]=0x021A -> line=0x003F -> line_cache=0x0046 -> tag=0 -> se imprime -> 1000011
z[8]=0x021C -> line=0x0040 -> line_cache=0x0046 -> tag=0 -> se imprime -> 1000011
z[9]=0x021E -> line=0x0040 -> line_cache=0x0046 -> tag=0 -> se imprime -> 1000011

```

**Exercici 4.2. Quants bits té line\_cache? Com escriurà en pantalla? Què escriu en cada fila? I en cada columna? Si creus que no tindràs suficient lloc per escriure a cada fila pots modificar els paràmetres, fil i col, adecuadament.**

La memòria cau té 512 línies, i això requereix 9 bits per representar l'índex de la línia. Per tant, line\_cache sempre conté 9 bits que identifiquen el número de línia de memòria cau.

A la pantalla, line\_cache es mostrarà com una seqüència de 9 bits (un '0' o '1' per columna) en les columnes especificades. A cada iteració, el codi imprimeix cada un dels bits de line\_cache des del bit més significatiu a la dreta, utilitzant l'operador line\_cache >> 1 per desplaçar els bits cap a la dreta i generar el següent bit per imprimir.

Els 9 bits es mostraran en les columnes especificades, i per a cada valor de X[i], Y[i] i Z[i], es mostraran 9 bits en una fila, de manera seqüencial. La lògica del bucle while(line\_cache != 0) només processa els bits significatius (els que són diferents de zero), per això, no es mostren mai els 9 bits sino que es mostren 6 o 7 bits.

Cada fila mostra els 9 bits de line\_cache de l'adreça de memòria corresponent a l'element X[i], Y[i] o Z[i] a la memòria. Aquests 9 bits es deriven de la línia de memòria calculada a partir de l'adreça de la variable (és a dir, line = address >> 3). Com he comentat abans, la lògica del bucle while(line\_cache != 0) només processa els bits significatius (els que són diferents de zero), per això, no es mostren mai els 9 bits sino que es mostren 6 o 7 bits..

Y, a cada columna, és mostra un sol bit (ja sigui '0' o '1') que correspon al valor d'un bit de line\_cache. Els bits de l'adreça de memòria es desplacen cap a la dreta (amb line\_cache = line\_cache >> 1) perquè es mostrin a les columnes corresponents de la pantalla.

No hem fet ninguna modificació a les files i columnes per defecte.

**Exercici 4.3.** Executa el codi anterior i fes una captura de pantalla (adjunta-la) del que has obtingut. Coincideix el resultat de line\_cache de cada accés del que obtindries a partir de les adreces de X, Y i Z. **IMPORTANT:** Al canviar el codi t'hauran canviat les posicions de memòria on es col·loquen X, Y, Z. Per poder extreure i comparar la línia i el TAG torna a obtenir les direccions inicial de X, Y, Z. Cada quantes dades canvia la linea de cache?

7

|        |         |         |
|--------|---------|---------|
| 111100 | 111111  | 1000001 |
| 111100 | 111111  | 1000001 |
| 111101 | 111111  | 1000010 |
| 111101 | 111111  | 1000010 |
| 111101 | 1000000 | 1000010 |
| 111101 | 1000000 | 1000010 |
| 111110 | 1000000 | 1000011 |
| 111110 | 1000000 | 1000011 |
| 111110 | 1000001 | 1000011 |
| 111110 | 1000001 | 1000011 |

El resultat de line\_cache coincideix en tots, el mateix valor que si ho obtindriem a partir de les adreces de X, Y i Z.

Observant la sortida i les adreces de memòria, es manté la mateixa linea\_cache durant 2 dades (x[0] a x[1]), després canvia i es manté durant 4 dades (x[2] a x[5]), després igual, canvia i és manté durant 4 dades (x[6] a x[9]), després, un altre cop canvia i es manté durant 4 dades (y[0] a y[3]), un altre cop canvia i es manté durant 4 dades (y[4] a z[7]), un altre cop canvia i es manté durant 4 dades (y[8] a z[11]), un altre cop canvia i es manté durant 4 dades (z[2] a z[5]), i per últim, es manté durant 4 dades (z[6] a z[9]).

**Exercici 4.4.** Observa també amb el simulador el tag que obtens per X, Y i Z. És el mateix per tots els accesos a X? i a Y? i a Z? Apunta aquí el resultat i compara'l amb el que obtindries a partir de les adreces inicials de X, Y i Z

Els tags per a totes les adreces de X[i], Y[i] i Z[i] són sempre iguals (0) dins de cada bloc de memòria (des de 0x01E4 a 0x021E). Quan canvia la línia de caché, el tag segueix sent el mateix per a totes les adreces dins del mateix bloc de memòria.